

## 6일 차

# 딥러닝 모델 성능 다루기



드롭아웃

- 과적합 방지 -> 일반화 성능 높
- 퍼센트로 줌.

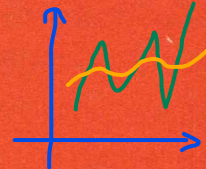
### 이번 차시에서 주목할 키워드

deep

- 모델 복잡도
- 과적합
- 과소적합 발생하면 안 됨.(학습 자체가 안 된 것)
- 조기 종료
- 드롭아웃
- 가중치 규제
- 일반화 성능

가울기 크면 가파라짐.

규제: 가파라서 뽀족해지는 걸 뭉개서 평평하게 만들



일반화 성능이  
좋아짐.

가울기  
가중치 Weight  
규제

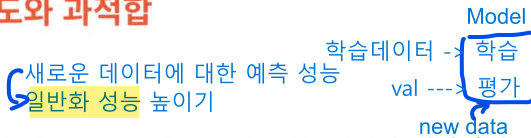
## 6일 차 학습 목표

- 모델의 복잡도와 과적합을 이해하고 과적합이 발생하는 이유를 설명할 수 있습니다.
- 과적합을 방지하기 위한 대표 기법의 원리를 이해하고 비교할 수 있습니다.
- 조기 종료, 드롭아웃, 가중치 규제를 적용해 모델의 성능을 개선할 수 있습니다.

딥러닝 모델은 유연하고 표현력이 강하지만, 그만큼 학습 데이터에 지나치게 맞춰져 새로운 데이터에서는 성능이 떨어지는 과적합(Overfitting) 문제가 자주 발생합니다. 이번 차시에서는 과적합 문제를 해결하고 모델의 일반화 성능을 높이는 방법, 즉 모델의 복잡도를 적절히 조절해 과적합을 회피하는 기법들을 살펴봅니다.

## 모델의 복잡도와 과적합

### 모델링의 목적



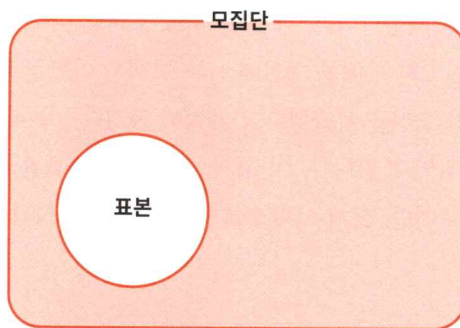
새로운 데이터에 대한 성능이 중요

실제 현장에서 발생하는 비즈니스 데이터에는 다양한 패턴이 숨어 있습니다. 머신러닝/딥러닝 모델을 만드는 궁극적인 목표는 데이터의 패턴을 학습해 새로운 데이터에서도 패턴을 정확히 예측하도록 만드는 일입니다. 쉽게 말해 모델링을 한다는 것은 과거 데이터에서 규칙을 찾아내고 그 규칙을 이용해 앞으로의 결과를 예측하는 과정입니다. 따라서 모델의 목적은 단순히 학습 데이터에서 높은 성능을 내는 데 그쳐서는 안 됩니다. 학습 과정에서 사용하지 않은 데이터(미래에 발생할 데이터, 운영 환경에서의 데이터)에서 안정적으로 좋은 결과를 내는 것이 보다 중요합니다. 새로운 데이터에서 작동하는 능력을 ‘일반화 성능’, 다른 말로는 ‘객관적 성능’이라고 합니다.

모델의 일반화 성능을 측정하기 위해 학습에서 사용하지 않았던 검증 데이터나 테스트 데이터로 객관적인 평가를 수행합니다.

모델을 학습할 때 사용하는 데이터는 전체 모집단(Population)에서 일부를 추출한 표본(Sample) 집합입니다. 모델은 표본 데이터에서 학습한 패턴을 전체 모집단 데이터에서도 적용할 수 있어야 합니다. 그러나 모델이 너무 단순하거나 반대로 지나치게 복잡하면 일반화 성능을 내기가 어렵습니다.

이 문제를 해결하려면 모델의 복잡도를 적절히 조정해 학습 데이터뿐만 아니라 새로운 데이터에서도 좋은 성능을 내도록 해야 합니다. 그렇다면 모델의 복잡도는 무엇일까요?



[그림 6-1] 모집단과 표본

### 모델의 복잡도와 일반화

모델이 좋은 성능을 내려면 복잡도를 알맞게 조정해야 합니다. 여기서 복잡도란 단순히 ‘복잡하다’는 뜻이 아니라, 학습 데이터의 패턴이 모델에 얼마나 반영되어 있는지를 의미합니다.

모델의 복잡도와 일반화 성능의 관계는 다음과 같이 정리할 수 있습니다.

- 너무 단순한 모델: 학습 데이터의 패턴을 제대로 학습하지 못한 상태입니다. 학습 데이터에서도 성능이 낮고 검증 데이터에서도 성능이 좋지 않습니다. 예를 들어 고객 이탈 예측 모델에서 “남성 고객은 이탈 가능성이 높다”는 단 하나의 패턴만을 반영해 남성 고객은 모두 이탈하고 여성 고객은 모두 유지된다고 예측 한다면 이는 지나치게 단순화한 판단입니다. 실제로는 나이, 구매 이력, 최근 활동 등 다양한 요인이 작용할 수 있는데, 단순한 패턴만을 학습하고 예측하면 학습 데이터에서 부정확한 결과를 내는 것은 물론 새로운 데이터에서도 일반화 성능을 기대하기 어렵습니다.
- 적절히 복잡한 모델: 학습 데이터에서 좋은 성능을 내면서 검증 데이터에서도 유사한 수준의 성능을 유지합니다. 즉, 학습 데이터에 과도하게 맞춰지지 않고 다양한 패턴을 적절히 반영해 새로운 데이터에서도 잘 작동하는 모델입니다. 이러한 모델은 일반화 성능이 좋으므로 운영 과정에서 신뢰할 수 있는 예측 결과를 기대할 수 있습니다.
- 너무 복잡한 모델: 학습 데이터에서는 매우 높은 성능을 보이지만, 검증 데이터에서는 성능이 오히려 떨어집니다. 이는 모델이 학습 데이터에 있는 세부 패턴과 노이즈(Noise)까지 모두 학습한 나머지 새로운 상황에서는 잘 대응하지 못하

는 과적합 상태이기 때문입니다. 예를 들어 고객 이탈 예측 모델이 “나이는 26세로 지난달 A 제품을 2회 구매한 수원에 사는 고객은 이탈할 확률이 높다”와 같은 지나치게 구체적인 조건을 학습했다고 가정해 봅시다. 이 조건은 특정 데이터에서만 적용되는 규칙일 뿐입니다. 이 고객 이탈 예측 모델은 비록 학습 데이터에서는 정확할지 모르지만, 실제 서비스에 적용하면 예외 상황에 취약해 예측력이 떨어질 겁니다.

노이즈란 데이터 안에 존재하는 불규칙한 값이나 우연히 발생한 오류입니다. 예측에 유의미한 정보가 아닌데도 불구하고 모델이 이를 실제 패턴으로 착각해 학습하면 과적합의 원인이 될 수 있습니다. 특히 모델이 너무 복잡하거나 학습을 오래 지속하면 이런 노이즈까지도 학습하게 되어 새로운 데이터의 일반화 성능이 떨어집니다. 따라서 노이즈는 가능한 한 제거하거나 모델이 노이즈에 민감하게 반응하지 않도록 복잡도를 조절하는 작업이 필요합니다.

### 과소적합과 과적합

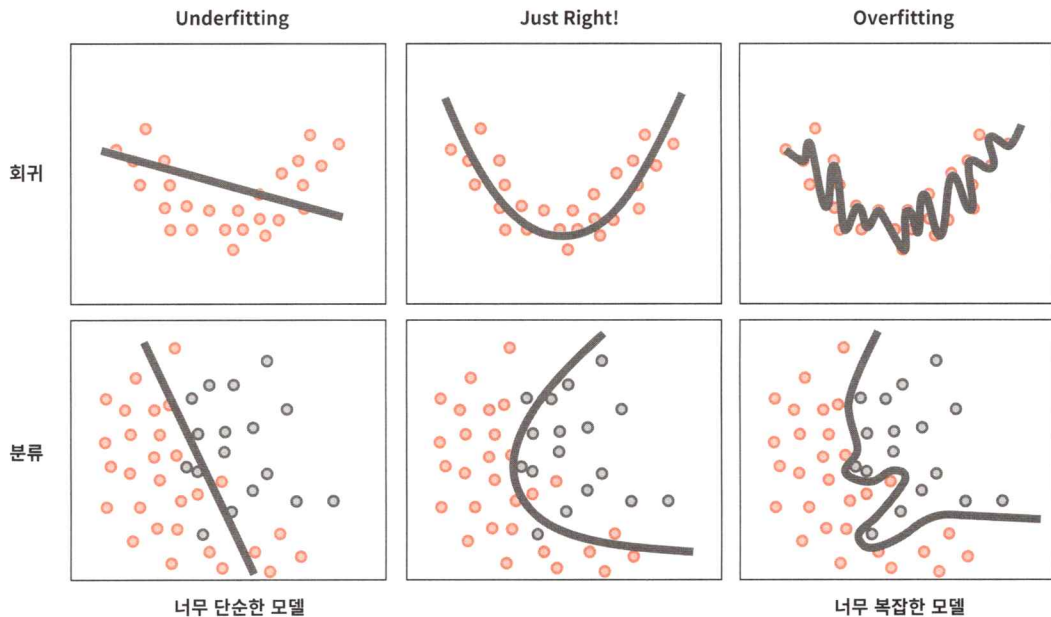
모델이 새로운 데이터를 잘 예측하려면 학습 과정에서 적절한 복잡도를 유지하는 일이 중요합니다. 모델이 너무 단순하면 데이터의 패턴을 제대로 학습하지 못해 성능이 낮아지고, 반대로 너무 복잡하면 학습 데이터에 지나치게 맞춰져 새로운 데이터에서는 오히려 성능이 떨어질 수 있습니다. 이러한 두 가지 문제를 각각 **과소적합(Underfitting)**과 **과적합(Overfitting)**이라고 합니다.

[그림 6-2]는 과소적합, 과적합이 무엇인지 보여 주는 그래프입니다.

**과소적합**      학습,검증,테스트 성능 다 안 좋음.

과소적합은 모델이 학습 데이터의 패턴을 충분히 학습하지 못한 상태입니다. 이 경우 모델은 학습 데이터에서 좋은 성능을 내지 못하며 당연히 새로운 데이터에서도 예측 성능이 떨어집니다. 과소적합은 주로 모델이 지나치게 단순할 때 발생합니다. 데이터의 다양한 패턴을 포착하는 능력이 부족한 경우라고 할 수 있습니다. 또한 중요한 특징(feature)을 잘 반영하지 못해 데이터에 숨겨진 구조를 놓치는 일도 하나의 원인입니다.

[그림 6-2]의 왼쪽 상단 그래프에서는 입력 데이터가 곡선 형태의 패턴을 가지고 있음에도 단순한 직선으로 학습하는 선형회귀 모델을 사용하고 있습니다. 이 경우



[그림 6-2] 과소적합과 과적합

데이터의 복잡한 구조를 제대로 반영하지 못해 예측 성능이 크게 떨어집니다. 그 하단의 분류 문제 그래프도 마찬가지입니다. 데이터의 분포상 곡선 형태의 경계가 필요함에도 불구하고 단순한 직선으로 두 범주를 나누려 할 경우, 많은 표본이 올바르게 분류되지 못해 오류가 발생할 수밖에 없습니다.

과소적합을 해결하려면 모델의 복잡도를 높이거나 더 많은 특징과 데이터를 학습에 활용해야 합니다. 예를 들면 은닉층(Hidden Layer) 또는 노드 수를 늘리거나 학습 시간을 충분히 확보하면 모델이 데이터의 패턴을 충분히 학습할 수 있습니다.

**과적합** 학습, 검증(validation)까지는 성능 잘 나오는데  
테스트(new data)에서는 성능 안 나옴. 일반화 성능 안 나옴.

과적합은 학습 데이터에 지나치게(Over) 맞춰져(Fitting) 새로운 데이터에서 예측력이 떨어지는 현상입니다. 모델이 학습 데이터의 패턴뿐만 아니라 노이즈까지 학습하는 경우에 발생합니다. 한마디로 과적합 모델은 학습 데이터를 하나도 틀리지 않고 다 맞추겠다고 학습하는 모델입니다. 따라서 학습 데이터에서는 높은 성능을 보이지만, 검증 데이터에서는 성능이 급격히 떨어지는 것이 특징입니다.

[그림 6-2]의 오른쪽 상단에 있는 회귀 그래프에서는 너무 복잡한 모델(Overfitting)을 사용한 나머지 모델이 학습 데이터의 작은 변화까지 따라가며 지나치게 세

밀한 곡선을 그리는 현상을 봅니다. 이 모델은 학습 데이터에서는 매우 높은 정확도를 보이지만, 새로운 데이터에서는 일반화하지 못해 오히려 예측 성능을 떨어뜨립니다. 그 하단의 분류 문제 그래프에서도 비슷한 현상이 나타나는데, 학습 데이터의 특정한 패턴까지 학습하기 때문에 검증 데이터에서 불필요하게 복잡한 결정 경계를 형성하는 경향이 있습니다.

문제:  
일반화 성능  
저하

## 모델의 복잡도 이해

과적합

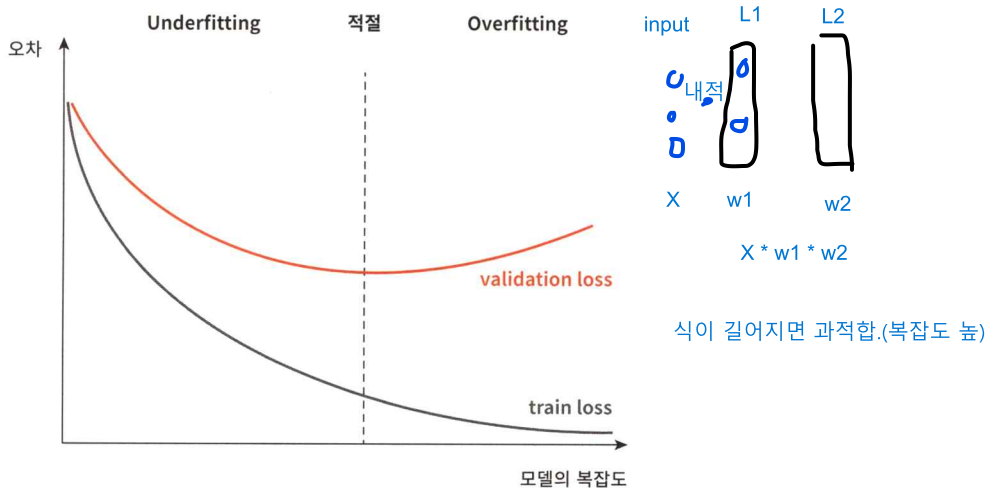
$$y = ax + b$$

$$y^1 = w1 * x1 + w0$$

$$y^2 = w2 * y^1 + w2$$

모델을 설계할 때 복잡도가 적절한 모델을 찾기 위해 노력해야 합니다. 그렇다면 적절한 복잡도의 모델을 어떻게 만들 수 있을까요? 어떤 부분이 모델 복잡도에 영향을 미칠까요?

[그림 6-3]은 모델의 복잡도에 따라 학습 오차(train loss)와 검증 오차(validation loss)가 어떻게 변화되는지 보여 주는 그래프입니다. 이 그래프를 이용하면 모델 복잡도의 적절한 지점을 찾을 수 있습니다.

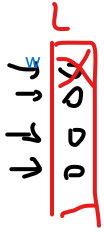


[그림 6-3] 모델 복잡도에 따른 오차 그래프

일반화 성능 높이기

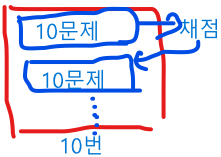
### 적절한 복잡도 지점 찾기

모델이 새로운 데이터에서 좋은 성능을 내려면 복잡도를 적절히 조절해야 한다고 했습니다. 하지만 어떤 복잡도가 가장 적절한지는 모델의 종류, 데이터의 특성에 따라 달라집니다. 이제 모델의 복잡도를 조절해 최적의 지점을 찾는 일반적인 방법들을 정리합니다.



예폭이 크면 클수록 똑같은 문제  
여러 번 풀어 외우게 됨.  
-> 새로운 문제에서 일반화가 떨어짐  
과적합..

100문제  
batch\_size=10



batch\_size=100이면 한 번 학습  
batch\_size=10이면 10문제 풀고  
채점해서 피드백 기반으로  
10문제 더 풀. 성능 더 높일 수 있음.  
batch\_size 적절하게 설정하기.

원래답 오차 data  
 $y = w_0 + w_1 \cdot x_1$   
 $y = b + a \cdot x_1$   
기울기

중요한 건 복잡도가 아닌  
일반화 성능.  
복잡해도 일반화 성능  
좋으면 됨.

- **모델별 복잡도 조정 요소**: 각각의 머신러닝 또는 딥러닝 모델은 고유한 방식으로 복잡도를 조절할 수 있습니다. 예를 들어 의사결정나무(Decision Tree)는 트리의 최대 깊이(Max Depth)를 조절할 수 있습니다. 깊이가 깊을수록 더 많은 조건을 분기하기 때문에 복잡한 모델이 됩니다. **신경망(Neural Network)**에서는 **은닉층의 수나 각 층의 노드 수를 조절함**으로써 모델의 복잡도를 바꿀 수 있습니다. 더 많은 층과 노드를 추가하면 모델은 복잡해지고 더 다양한 패턴을 학습할 수 있습니다.

- 하이퍼파라미터 튜닝: 모델의 복잡도를 조절하는 또 다른 일반적인 방법은 하이퍼파라미터(Hyperparameter)를 튜닝하는 것입니다. 하이퍼파라미터는 학습하기 전에 설정하는 값으로 모델의 학습 과정과 구조에 큰 영향을 미칩니다. 예를 들어 **학습률(Learning Rate)**, **에포크(Epoch) 수**, **배치 크기(Batch Size)**, 드롭아웃 비율(Dropout Rate), **규제 강도( $\lambda$ )** 등은 모두 모델의 학습과 복잡도에 영향을 미치는 하이퍼파라미터입니다. 이 값들을 적절히 조합하는 일을 하이퍼파라미터 튜닝이라고 하는데, 일반적으로 실험을 통해 검증 데이터를 반복적으로 비교하는 방식으로 최적의 조합을 찾습니다.

- 오차 비교를 통한 최적화: 그래프로 모델이 현재 과소적합인지 과적합인지 아니면 적절한 복잡도를 갖추고 있는지를 적절히 판단할 수 있습니다. [그림 6-3]에서 학습 오차와 검증 오차가 줄어들지 않고 값이 모두 높으면 모델이 너무 단순해서 데이터를 충분히 설명하지 못하는 상태입니다. 또한 학습 오차는 줄어드는데, 검증 오차는 다시 늘어난다면 모델이 학습 데이터에 과도하게 맞춰져 새로운 데이터에서 일반화되지 못하는 상태입니다. 끝으로 학습 오차와 검증 오차가 줄어들어 값이 모두 낮고 두 그래프의 흐름도 비슷하다면 모델이 학습 데이터의 핵심 패턴을 잘 학습했고 검증 데이터도 잘 일반화한 상태입니다.

모델의 복잡도는 너무 낮아도 또는 너무 높아도 문제가 됩니다. [그림 6-3]의 그래프로 오차의 흐름을 관찰하고 다양한 하이퍼파라미터를 조정하면서 적절한 복잡도 지점을 찾아가는 일이 모델 성능 최적화의 핵심입니다.

#### 딥러닝 모델의 복잡도와 파라미터 수

딥러닝 모델의 복잡도를 대표하는 지표가 바로 파라미터(Parameter) 수입니다. 파라미터는 학습 과정에서 데이터에 담긴 패턴을 찾아내고 이를 수치화한 값입니다.

쉽게 말해 파라미터는 모델이 데이터를 이해하고 예측하기 위해 기억해 둔 ‘지식’이라고 할 수 있습니다.

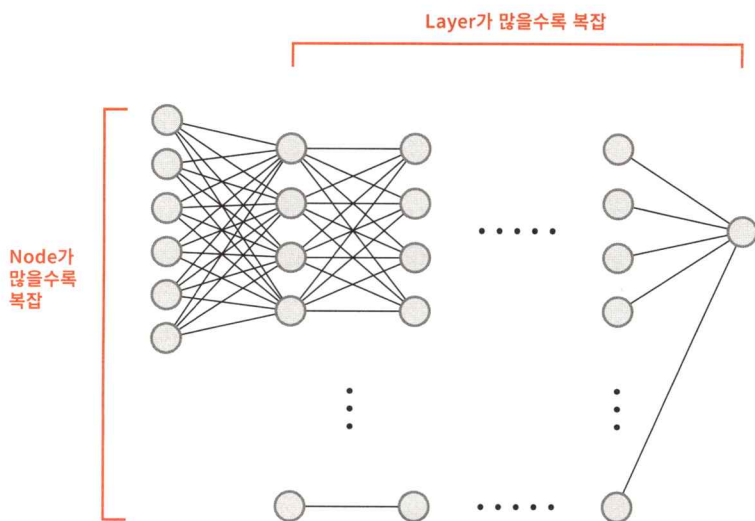
하지만 파라미터가 지나치게 많으면 문제가 생길 수 있습니다. 파라미터가 많을수록 모델은 데이터의 **세세한 부분까지 학습**하는데, 이는 **과적합**으로 이어질 가능성이 높아집니다. 또한 파라미터가 늘어나면 연산량이 증가해 학습과 추론 속도가 느려지고 더 많은 메모리를 사용하므로 시스템 비용 역시 커집니다.

중요도  
성능 > 비용

파라미터 수는 **입력 데이터의 크기, 은닉층의 개수, 각 층의 노드(뉴런) 수에 따라 결정**됩니다. 예를 들어 입력 특징(feature)이 많아지거나 은닉층을 여러 개 추가하거나 각 층에 노드를 많이 배치하면 파라미터 수는 급격히 증가합니다. 따라서 모델을 설계할 때는 단순히 큰 모델을 만드는 것보다 적절한 규모에서 높은 성능을 내는 최적의 구조를 찾는 일이 중요합니다.

실제로 딥러닝 프로젝트의 모델 설계에는 정답이 없습니다. 데이터 특성, 문제의 난이도, 사용할 수 있는 연산 자원에 따라 적절한 파라미터 규모가 달라집니다. 따라서 다양한 조합으로 실험을 반복하면서 성능과 효율성의 균형을 찾는 과정이 필요합니다.

[그림 6-4]는 복잡도에 영향을 주는 요소들을 그림으로 표현하고 있습니다. 일반적으로 모델 구조에서 층(Layer)과 노드 수가 많을수록 모델은 점점 복잡해집니다.



[그림 6-4] 딥러닝에서 모델 복잡도에 영향을 주는 요소

## 딥러닝의 모델 복잡도 조절하기

이번에는 모델 복잡도에 영향을 주는 대표적인 요소들을 하나씩 살펴보며 어떤 점을 고려해 설계해야 하는지 설명하겠습니다.

### 은닉층

은닉층은 입력층과 출력층 사이에 위치한 층으로 데이터를 단계적으로 변환하면서 의미 있는 특징(feature)을 자동으로 추출하는 역할을 합니다. 이 과정에서 모델은 원시 입력 데이터를 점차 정제해 문제 해결에 유용한 정보로 바꾸어 갑니다.

은닉층이 많아질수록 모델은 더 복잡하고 추상적인 패턴(Pattern)을 학습할 수 있는 표현력을 지닙니다. 예를 들어 하나의 은닉층이 있는 단순한 신경망은 직선이나 곡선 정도의 단순한 패턴을 학습할 수 있지만, 여러 개의 은닉층을 쌓은 심층 신경망(Deep Neural Network)은 이미지에 있는 사람의 얼굴을 인식하거나 자연어의 문맥을 이해하는 등 훨씬 복잡한 문제도 해결할 수 있습니다. 즉, 은닉층은 데이터를 단순히 처리하는 층이 아니라, 특징을 추출하고 이를 기반으로 유의미한 패턴을 학습하는 핵심 구조로서 층이 깊을수록 모델은 더욱 정교하고 풍부한 표현을 할 수 있습니다.

하지만 은닉층을 무작정 많이 추가한다고 능사는 아닙니다. 층이 늘어날수록 파라미터 수가 폭발적으로 증가해 과적합 위험이 커지고 학습 시간이 오래 걸리며 연산 자원이 많이 소모됩니다. 또한 층이 너무 깊으면 기울기 소실(Vanishing Gradient) 같은 학습 문제도 발생할 수 있습니다.



### 기울기 소실 문제가 뭐예요?

기울기 소실(Vanishing Gradient) 문제는 딥러닝 모델을 학습할 때 층이 깊어질수록 역전파(Backpropagation) 과정에서 전달되는 기울기(Gradient) 값이 점점 작아져 결국 0에 가까워지는 현상을 말합니다. 딥러닝은 가중치(Weight)를 조금씩 조정하며 학습을 진행하는데, 기울기가 0에 가까워지면 가중치가 거의 업데이트되지 않습니다. 가중치가 업데이트되지 않으면 모델은 학습을 제대로 진행할 수 없고, 특히 앞쪽(입력층 가까이)에 있는 층들은 학습을 거의 멈춥니다.

최근에는 ReLU(Rectified Linear Unit)와 같이 기울기 소실을 완화하는 활성화 함수를 사용하거나 다른 기법을 사용해 문제를 해결합니다.

## 각 층의 노드 수

각각의 은닉층에 배치되는 노드(뉴런) 수는 해당 층이 학습할 수 있는 정보의 양을 결정합니다. 노드가 많으면 더 다양한 특징을 학습할 수 있고 복잡한 데이터를 잘 표현할 가능성이 커집니다. 예를 들어 노드가 적은 모델은 이미지에서 단순한 윤곽선만 인식할 수 있지만, 노드를 충분히 늘리면 색상, 질감, 세부 패턴까지 포착할 수 있습니다.

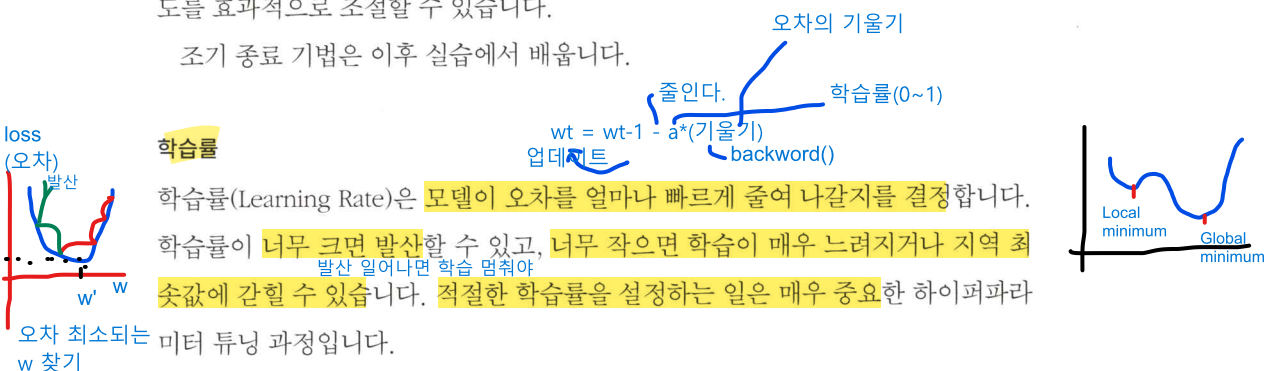
하지만 노드 수가 많아지면 파라미터 수도 함께 증가해 연산량이 커지고 메모리 사용량도 늘어납니다. 결국 모델의 복잡도는 급격히 상승하는데, 데이터가 충분하지 않다면 오히려 과적합이 발생할 수 있습니다. 따라서 노드 수는 단순히 크게 잡기보다는 데이터의 특성과 문제 난이도에 따라 실험을 거쳐 최적의 값을 찾아야 합니다.

## 에포크 과적합과는 관련 있지만 파라미터 수와는 관련x

에포크는 모델이 전체 학습 데이터를 몇 번 반복해서 학습할지를 정하는 값입니다. 에포크가 너무 적으면 모델이 데이터의 패턴을 충분히 배우지 못해 학습이 덜 된 상태로 끝나는 과소적합이 발생합니다. 반대로 에포크를 지나치게 크게 설정하면 모델이 학습 데이터의 세부적인 노이즈까지 학습해 과적합될 수 있습니다.

일반적으로 모델 학습 과정에서는 검증 데이터를 활용해 에포크가 어느 시점에서 가장 좋은 성능을 내는지 관찰하며, 적절한 시점에 학습을 멈추는 **조기 종료 (Early Stopping)** 기법을 함께 사용합니다. 조기 종료 기법들을 사용하면 모델 복잡도를 효과적으로 조절할 수 있습니다.

조기 종료 기법은 이후 실습에서 배웁니다.



## 학습률

학습률(Learning Rate)은 모델이 오차를 얼마나 빠르게 줄여 나갈지를 결정합니다.

학습률이 너무 크면 발산할 수 있고, 너무 작으면 학습이 매우 느려지거나 지역 최소값에 갇힐 수 있습니다. 적절한 학습률을 설정하는 일은 매우 중요한 하이퍼파라

모델의 은닉층 수, 노드 수, 에포크, 학습률 등 다양한 요소들이 모델의 복잡도를 결정합니다. 복잡도가 적절한 모델을 찾기 위해서는 다양한 설정을 실험해 보는 과

정, 즉 하이퍼파라미터 튜닝이 필요합니다.

하지만 하이퍼파라미터 튜닝 과정은 생각보다 훨씬 복잡하고 시간이 오래 걸릴 수 있습니다. 예를 들어 은닉층의 수를 1부터 5까지 바꾸고 노드 수도 5, 10, 15, 20, 25로 조정한다고 가정해 봅시다. 이 경우 은닉층의 경우의 수 5와 노드 수의 경우의 수 5를 곱하면 총 25번의 실험이 필요합니다. 여기에 에포크 수를 25, 50, 100, 200으로 바꿔가며 학습하면 25번의 실험을 각각 4번 반복해야 하므로 총 100번의 실험을 해야 합니다. 만약 학습률까지 조정한다면 실험 횟수는 기하급수적으로 늘어나게 됩니다.

### 과적합 규제 기법들

다양한 하이퍼파라미터를 조정해 최적의 조합을 찾는 일은 매우 중요하지만, 동시에 매우 어렵고 복잡한 작업입니다. 그래서 딥러닝에서는 복잡도 조절 외에도 모델이 과적합되는 것을 사전에 방지하는 여러 규제 기법을 함께 사용합니다.

- **조기 종료**(Early Stopping): 에포크가 거듭될수록 검증 데이터에서 성능이 더 이상 개선되지 않으면 학습을 조기에 종료함으로써 과적합을 방지합니다. 학습을 오래 진행할수록 학습 성능은 좋아지지만, 검증 성능이 떨어질 수 있으므로 이를 방지하는 데 효과적입니다.
- **드롭아웃**(Dropout, 임의로 연결 끊기): 학습 과정에서 일부 노드를 무작위로 비활성화시켜 모델이 특정 노드나 경로에 과도하게 의존하지 않도록 합니다. 이렇게 함으로써 다양한 경로에서 들어오는 데이터 조합으로 학습할 수 있어 **일반화 성능이 향상**됩니다.
- **가중치 규제**(Regularization): 손실 함수의 가중치에 페널티 항을 추가해 가중치가 너무 커지지 않도록 제한합니다. 대표적으로 L1 규제와 L2 규제가 있으며 이 규제로 **모델의 복잡도를 간접적으로 조절**합니다.

이 기법들은 복잡한 하이퍼파라미터 실험 없이도 모델이 적절한 수준의 복잡도를 유지하도록 도와줍니다. 다음 절에서는 이들 규제 기법을 하나씩 살펴봄과 과적합을 방지하고 일반화 성능을 높이는 방법을 실습과 함께 살펴보겠습니다.

3개를 다 합쳐보는 것도 필요.

## 모델 성능 최적화 실습 1: 과적합 모델

이번 실습에서는 성능 최적화 실습을 위해 모델 구조와 하이퍼파라미터를 조정해 과적합이 발생한 모델을 먼저 생성합니다. 구현 절차는 환경 준비, 데이터 전처리, 모델링 및 학습 곡선 확인 순서로 진행합니다.

### 환경 준비

새 코랩 파일을 생성하고 파일 이름은 '6일차\_모델성능최적화.ipynb'로 명명합니다.

### 런타임 유형 변경

이번 실습에서는 학습 시간이 많이 소요될 수 있으므로 GPU를 사용합니다. 코랩 파일 상단 메뉴에서 [런타임] - [런타임 유형 변경]을 누른 다음, 나온 [런타임 유형 변경] 대화상자에서 하드웨어 가속기 유형을 'T4 GPU'로 선택합니다.

이어진 환경 준비에서는 딥러닝에 필요한 라이브러리를 불러오고 사용자 정의 함수 4가지(데이터 로더 함수, 학습 함수, 평가 함수, 학습 곡선 함수)를 생성합니다. 또한 디바이스도 준비해야 합니다. 이미 여러 번 반복 작성한 코드이므로 여러분은 5일 차를 참조해 하나씩 순서대로 입력하고 실행합니다.

### 데이터 불러오기: Fashion-MNIST

이번 실습에서는 Fashion-MNIST가 어떤 구조로 이루어졌는지 데이터를 살펴보는 과정에서 시작합니다.

Fashion-MNIST 데이터 세트는 의류와 패션 아이템 이미지로 구성된 딥러닝용 공개 데이터 세트입니다. Fashion-MNIST 데이터 세트는 MNIST 데이터 세트처럼 28×28 픽셀 크기의 흑백 이미지 70,000개를 제공하지만, 숫자 이미지가 아닌 티셔츠, 바지, 스니커즈, 가방 등 10개 범주의 의류 이미지들로 구성되어 있습니다. 각각의 이미지는 하나의 아이템을 나타냅니다. 70,000개의 데이터 세트는 학습용 60,000개, 테스트용 10,000개로 구성되어 있습니다. MNIST보다 조금 더 복잡한 이미지 패턴을 포함하고 있어 딥러닝 분류 모델을 실습하기에 좋은 데이터입니다.

#### CODE

```
from torchvision import datasets
from torchvision.transforms import ToTensor
```

```
train_dataset = datasets.FashionMNIST(root='data', train=True, download=True,
                                       transform=ToTensor())
test_dataset = datasets.FashionMNIST(root='data', train=False, download=True,
                                       transform=ToTensor())
```

코드는 5일 차에서 사용했던 MNIST 데이터 불러오기와 거의 동일합니다. 다만 이번에는 숫자로 쓴 손글씨 이미지(MNIST)가 아니라, 의류 이미지 데이터(Fashion-MNIST)라는 차이가 있을 뿐입니다.

학습 데이터는 `train=True`, 테스트 데이터는 `train=False` 옵션으로 설정해 불러옵니다. `transform=ToTensor`로 이미지를 텐서 형식으로 변환합니다. Fashion-MNIST 데이터는 코드를 실행하면 자동으로 다운로드되며 이미지들은 `root` 옵션에서 지정한 경로(data 폴더)에 저장됩니다.

모두 불러왔다면 10개의 의류 범주를 리스트에 저장하고 내용을 확인합니다.

#### CODE

```
class_list = train_dataset.classes
class_list
```

#### OUTPUT

```
['T-shirt/top',
 'Trouser',
 'Pullover',
 'Dress',
 'Coat',
 'Sandal',
 'Shirt',
 'Sneaker',
 'Bag',
 'Ankle boot']
```

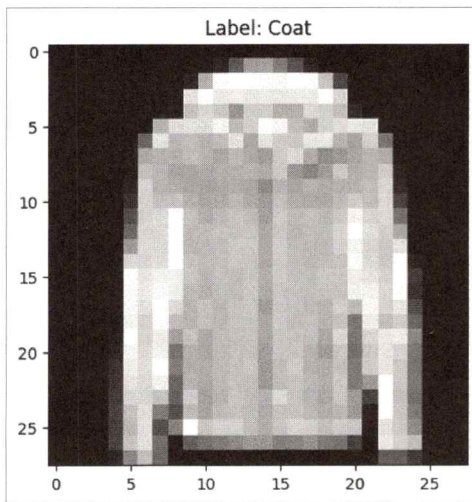
코드를 실행하면 10종의 의류 아이템 목록을 보여 줍니다.

계속해서 이번에는 이미지를 하나 출력하겠습니다.

#### CODE

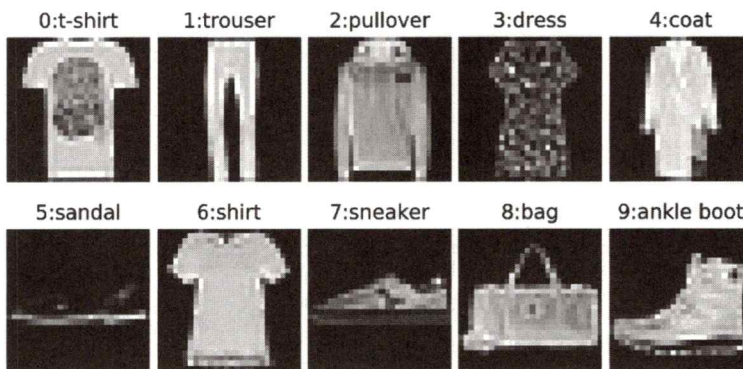
```
n = 5000
image, label = train_dataset.data[n], train_dataset.targets[n]

# 이미지 시각화
plt.imshow(image, cmap="gray")
plt.title(f"Label: {class_list[label]}")
plt.show()
```



코드 이미지를 하나 불러왔습니다. 코드에서 변수  $n$ 의 숫자를 변경하면서 어떤 이미지들이 저장되어 있는지 확인하기 바랍니다.

[그림 6-5]는 Fashion-MNIST 데이터 세트에서 제공하는 10가지 의류 범주의 이미지들입니다.



[그림 6-5] Fashion-MNIST 데이터 세트 이미지

이미지를 보고 각각의 범주로 분류해야 하는데, 이미지의 해상도( $28 \times 28$ )가 너무 떨어져 분류하기가 쉽지는 않습니다. 특히 t-shirt와 shirt는 사람이 봐도 헷갈릴 수 있습니다.

## 데이터 전처리

학습 데이터 60,000건을 그대로 이용하면 학습 시간이 많이 걸립니다. 그래서 데이터를 20,000건으로 줄인 과적합 모델을 생성할 예정입니다.

데이터 전처리는 다음과 같은 단계로 진행합니다.

1. 다운로드한 데이터 세트의 원본 데이터 추출 및 스케일링
2. 학습 데이터의 크기 줄이기: 과적합과 규제 기법을 실험할 때 학습 시간을 줄이기 위해 데이터의 크기를 줄임
3. 테스트 데이터 세트를 검증(val), 테스트(test) 데이터로 분할
4. 이미지 데이터 세트의 차원 맞추기(3차원 → 4차원)
5. 학습 데이터를 이용해 데이터 로더 구성

전처리 절차가 꽤 복잡해 보이지만, 코드는 비교적 간단합니다. 하나씩 살펴보겠습니다.

### 데이터 세트의 원본 데이터 추출 및 스케일링

다운로드한 두 개의 데이터 세트(train\_dataset, test\_dataset)에서 입력 데이터와 목표값 데이터를 추출하고 스케일링을 수행합니다.

#### CODE

```
x_train = train_dataset.data / 255 ①  
y_train = train_dataset.targets      ②  
x_test = test_dataset.data / 255  
y_test = test_dataset.targets
```

① 다운로드한 train\_dataset의 data(입력 데이터)를 255로 나누어 스케일링합니다.

② 다운로드한 train\_dataset의 targets(목표값 데이터)를 저장합니다.

이미지는 0~255 사이의 숫자 데이터로 구성되어 있습니다. 따라서 단순히 255로 나누는 것만으로도 MinMax 스케일링(0~1로 변환)이 적용됩니다.

### 학습 데이터의 크기 줄이기, 테스트 데이터 분할

학습 데이터가 크면(많으면) 학습 시간이 오래 걸리므로 다양한 실험을 하기가 어렵습니다. 그래서 데이터를 분할해 데이터의 크기를 줄입니다.

**CODE**

```
x_train = x_train[:20000]
y_train = y_train[:20000]
print(x_train.shape)
```

**OUTPUT**

```
torch.Size([20000, 28, 28])
```

학습 데이터 `x_train`과 `y_train`에서 20,000건만 뽑아 다시 `x_train`과 `y_train`에 저장합니다(덮어쓰기). 코드를 실행해 `x_train`의 구조를 보면  $28 \times 28$  크기의 이미지가 20,000건으로 변경되었습니다.

다음으로 10,000건의 테스트 데이터 세트를 반으로 나누어 검증(`val`)과 테스트(`test`) 데이터로 각각 저장합니다.

**CODE**

```
x_val, x_test = x_test[:5000], x_test[5000:]
y_val, y_test = y_test[:5000], y_test[5000:]
print(x_val.shape, x_test.shape)
```

**OUTPUT**

```
torch.Size([5000, 28, 28]) torch.Size([5000, 28, 28])
```

`x_val`, `x_test`를 각각  $[5000, 28, 28]$ 로 구성했습니다. 이렇게 해서 학습, 테스트, 검증 데이터 세트에 대한 기본적인 준비를 모두 마쳤습니다.

이어서 이미지 데이터의 차원을 맞추고 데이터 로더를 구성합니다.

### 이미지 데이터의 차원 맞추기, 데이터 로더 구성

`torchvision.datasets` 모듈에서 제공하는 MNIST나 FashionMNIST 데이터 세트의 경우 내부의 원본 이미지 데이터(`train_dataset.data`)를 추출하면 각각의 이미지는 2차원( $28 \times 28$ ) 흑백 이미지 상태로 저장되어 있습니다. 이미지의 채널(컬러) 차원이 생략되어 있기 때문에 딥러닝 모델에서 기대하는 표준적인 이미지 데이터 세트(4차원 텐서) 형태로 데이터를 변환해 주어야 합니다.

이미지 데이터 세트의 표준 구조(배치 크기, 채널, 세로 픽셀 수, 가로 픽셀 수)는 5일 차에서 설명한 내용을 참고하면 됩니다(5일 차 204~205쪽 참조). 다음과 같이 4차원 데이터 구조로 변경합니다.

**CODE**

```
x_train = x_train.view(20000, 1, 28, 28)
x_val = x_val.view(5000, 1, 28, 28)
x_test = x_test.view(5000, 1, 28, 28)
```

view 메서드를 이용해 [5000, 28, 28]의 3차원을 [5000, 1, 28, 28]의 4차원 데이터로 변환합니다. 두 번째 축 항목에 흑백 채널 1개를 추가했습니다.

이제 학습 데이터 세트(x\_train, y\_train)를 텐서 데이터 세트로 변환하고 데이터 로더를 구성합니다.

**CODE**

```
train_TensorDS = TensorDataset(x_train, y_train)
batch_size = 64
train_loader = DataLoader(train_TensorDS, batch_size=batch_size)
```

데이터 준비가 모두 끝났으므로 과적합 모델을 설계하고 학습 곡선으로 확인합니다.

## 과적합 모델 만들기

모델을 복잡하게 설계하고 에포크도 필요 이상으로 지정해 과적합 모델을 만듭니다.

### 복잡한 모델 구조

모델을 복잡하게 설계하려면 은닉층 수와 노드 수를 과도하게 늘리면 됩니다. 이번 예제에서는 은닉층은 4개, 층마다 노드 수는 512→256→256→64개순으로 지정하겠습니다. 이 구조보다 더 복잡하게 설계할 수 있지만, Fashion-MNIST 데이터 세트에서는 이 정도면 충분합니다.

다음과 같이 코드를 입력합니다.

**CODE**

```
n_feature = 28 * 28
n_class = 10

# 모델 구조 설계
model = nn.Sequential(nn.Flatten(),
                      nn.Linear(n_feature, 512), nn.ReLU(),
                      nn.Linear(512, 256), nn.ReLU(),
                      nn.Linear(256, 256), nn.ReLU(),
                      nn.Linear(256, 64), nn.ReLU(),
                      nn.Linear(64, n_class)
                      ).to(device)
```

```
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)
```

모델 구조를 살펴보면 먼저 입력 이미지를 Flatten 함수를 사용해 1차원 벡터 형태로 펼친 다음, 첫 번째 은닉층(노드 수 512)으로 연결합니다. 계속해서 은닉층 3개가 순차적으로 연결되는데, 각 층의 노드 수는 각각 256, 256, 64개로 설정되어 있습니다. 마지막 출력층은 총 10개의 범주를 예측하므로 노드 수는 10으로 지정했습니다.

설계한 모델의 요약 정보를 살펴보겠습니다. torchsummary 라이브러리의 summary 함수를 사용합니다.

#### CODE

```
from torchsummary import summary
summary(model, input_size = (n_feature,))
```

#### OUTPUT

| Layer (type)                          | Output Shape | Param # |
|---------------------------------------|--------------|---------|
| Flatten-1                             | [-1, 784]    | 0       |
| Linear-2                              | [-1, 512]    | 401,920 |
| ReLU-3                                | [-1, 512]    | 0       |
| Linear-4                              | [-1, 256]    | 131,328 |
| ReLU-5                                | [-1, 256]    | 0       |
| Linear-6                              | [-1, 256]    | 65,792  |
| ReLU-7                                | [-1, 256]    | 0       |
| Linear-8                              | [-1, 64]     | 16,448  |
| ReLU-9                                | [-1, 64]     | 0       |
| Linear-10                             | [-1, 10]     | 650     |
| Total params: 616,138                 |              |         |
| Trainable params: 616,138             |              |         |
| Non-trainable params: 0               |              |         |
| Input size (MB): 0.00                 |              |         |
| Forward/backward pass size (MB): 0.02 |              |         |
| Params size (MB): 2.35                |              |         |
| Estimated Total Size (MB): 2.38       |              |         |

summary 함수의 실행 결과는 총 세 부분으로 나누어져 있습니다. 요약 정보는 상단부터 모델 구조, 모델 크기(파라미터 관점), 모델 크기(용량 관점)순으로 보여 줍니다. 이 중 모델 크기에 있는 'Total params'가 이 모델의 전체 파라미터 수입니다. 방금 설계한 모델의 파라미터 수는 무려 616,138개입니다.

## 학습

이 모델을 `epochs=40`으로 학습시킵니다. MNIST나 Fashion-MNIST 데이터 세트는 보통 5~20에포크 정도면 충분하다고 알려져 있습니다. 하지만 여기서는 과적합 모델을 생성하기 위해 좀 더 과도한 에포크 수로 학습시킵니다.

빈 셀에서 다음과 같이 학습과 학습 곡선 코드를 입력합니다.

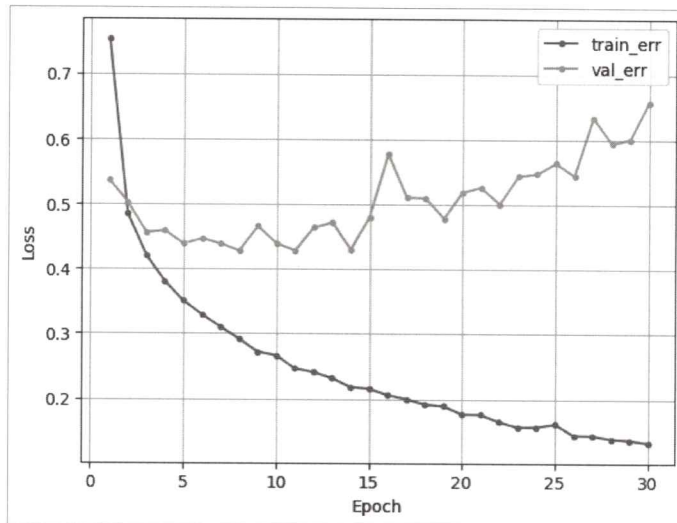
### CODE

```
epochs = 40
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, pred = evaluate(x_val, y_val, model, loss_fn, device)
    # 리스트에 추가
    tr_loss_list.append(tr_loss)    # train - CrossEntropy
    val_loss_list.append(val_loss)  # val - CrossEntropy
    print(f"Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}")

# 학습 곡선
dl_learning_curve(tr_loss_list, val_loss_list)
```

### OUTPUT

```
Epoch 1, train loss : 0.7568, val loss : 0.5555
Epoch 2, train loss : 0.4731, val loss : 0.4747
Epoch 3, train loss : 0.4181, val loss : 0.4463
...
Epoch 38, train loss : 0.1088, val loss : 0.6541
Epoch 39, train loss : 0.1034, val loss : 0.6445
Epoch 40, train loss : 0.1053, val loss : 0.7337
```



코드의 실행 결과를 보면 학습 초기에는 val\_loss(검증 오차)가 점차 줄어들면서 모델이 데이터를 잘 학습하고 있는 모습을 보여 줍니다. 예를 들어 에포크 1~3구간에서는 검증 오차가 0.5555→0.4747→0.4463으로 꾸준히 낮아지고 있습니다. 이는 모델이 학습 데이터를 기반으로 점차 일반적인 패턴을 파악하고 있다는 뜻입니다.

그러나 마지막 에포크 40에서는 검증 오차가 0.7337로 오히려 처음 학습을 할 때보다 더 나빠진 상태입니다. 검증 오차는 에포크 4~9 근처에서 최저점을 찍은 다음, 다시 증가하는 U자 형태의 곡선을 보입니다. 최저점에 도달하고 나서 모델은 학습 데이터에 과도하게 맞춰지면서(과적합) 검증 데이터에는 잘 대응하지 못하는 상태로 진입한 것입니다. 이와 같이 학습 오차는 계속 개선되지만, 검증 오차는 오히려 악화되는 현상이 나타나면 과적합(Overfitting)이 발생했다고 판단합니다.

## 모델 예측 및 평가

과적합 모델을 생성했으니 모델의 성능도 살펴보겠습니다. 빈 셀에서 다음과 같이 작성합니다.

### CODE

```
# 예측하고 소프트맥스 함수 적용
_, pred = evaluate(x_test, y_test, model, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)

# 가장 확률이 높은 인덱스 저장
pred = np.argmax(pred.cpu().numpy(), axis = 1) ①

# 모델 평가
print(classification_report(y_test, pred, target_names=class_list, digits=4)) ②
```

### OUTPUT

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| T-shirt/top  | 0.7984    | 0.8357 | 0.8167   | 493     |
| Trouser      | 0.9920    | 0.9538 | 0.9725   | 519     |
| Pullover     | 0.8067    | 0.7495 | 0.7771   | 479     |
| Dress        | 0.8656    | 0.9020 | 0.8834   | 500     |
| Coat         | 0.7533    | 0.8288 | 0.7893   | 479     |
| Sandal       | 0.9956    | 0.8757 | 0.9318   | 515     |
| Shirt        | 0.7091    | 0.6351 | 0.6701   | 518     |
| Sneaker      | 0.8611    | 0.9920 | 0.9219   | 500     |
| Bag          | 0.9452    | 0.9831 | 0.9638   | 474     |
| Ankle boot   | 0.9605    | 0.9293 | 0.9446   | 523     |
| accuracy     |           |        | 0.8684   | 5000    |
| macro avg    | 0.8688    | 0.8685 | 0.8671   | 5000    |
| weighted avg | 0.8699    | 0.8684 | 0.8676   | 5000    |

- ① GPU 디바이스에서 계산된 결과이므로 pred를 CPU로 옮긴 다음, 넘파이 배열로 변환해 np.argmax 함수에 입력합니다.
- ② classification\_report 함수에 추가한 digits=4는 결과를 소수점 4자리까지 표시하라는 의미입니다. 좀 더 정밀하게 보기 위함입니다.

전체 성능에 해당하는 정분류율은 0.8684입니다. 정밀도와 재현율의 평균(조화평균)인 F1 score 관점에서 각 범주의 성능을 살펴보면 모델은 'Trouser'에서 성능이 가장 높고(0.9725), 'Shirt'에서 성능이 가장 낮습니다(0.6701).

이어지는 절에서 과적합을 방지하는 다양한 방법을 설명합니다.

## 모델 성능 최적화 실습 2: 조기 종료

과적합을 방지하기 위해 사용하는 가장 직관적인 방법 중 하나가 조기 종료(Early Stopping)입니다.

### 조기 종로의 개념과 동작 원리

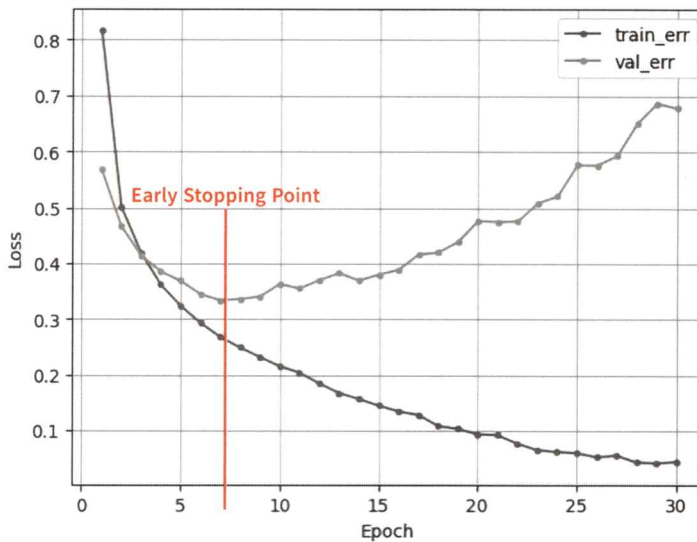
조기 종료는 학습 과정에서 검증 오차가 더 이상 줄어들지 않는 시점에 학습을 미리 끝내는 방법입니다. 조기 종료하면 모델이 학습 데이터를 지나치게 따라가는 현상을 막아 일반화 성능을 높일 수 있습니다.

일반적으로 모델 학습 과정에서 에포크 수가 증가하면 학습 오차는 계속 감소합니다. 이는 모델이 점점 더 학습 데이터에 최적화되기 때문으로 꼭 좋은 현상은 아닙니다.

[그림 6-6]의 그래프를 보며 조기 종로의 필요성을 설명하겠습니다.

- 초기에는 학습 데이터와 검증 데이터의 성능이 모두 향상됩니다.
- 특정 에포크부터 학습 데이터의 성능은 계속 좋아지지만, 검증 데이터의 오차는 오히려 다시 증가합니다. 이 지점이 바로 과적합(Overfitting)이 발생하는 시점입니다.

이때 검증 데이터의 오차가 더 이상 줄어들지 않으면 학습을 멈추는 것이 좋습니다. 학습을 멈추면 모델이 학습 데이터에 과도하게 최적화되는 것을 막을 수 있습니다.



[그림 6-6] 에포크 수와 조기 종료 지점 확인

조기 종료의 기본 원리는 간단합니다. 학습 과정에서 일정 에포크 동안 검증 오차가 개선되지 않으면 학습을 종료합니다. 조기 종료의 절차는 다음과 같습니다.

1. 학습이 진행되는 동안 에포크마다 검증 오차(Validation Error)를 측정합니다. 여기서 초점은 학습 오차(Train Error)가 아니라 검증 오차에 있음을 주의합니다.
2. 에포크마다 검증 오차를 비교하고 현재의 오차가 기존의 오차보다 낮으면 최적 오차(best\_loss)로 기록해 저장합니다.
3. 이후 에포크에서 측정한 검증 오차가 최적 오차(best\_loss)와 비교해 개선되지 않으면 카운트합니다. 카운트는 검증 오차가 개선되지 않은 연속된 횟수(p)입니다.
4. 일정 횟수(p) 동안 오차가 개선되지 않으면 학습을 중지합니다(조기 종료).

이 과정으로 불필요한 추가 학습을 방지해 모델이 과적합되는 것을 막습니다.

## 조기 종료 모델

조기 종료 모델을 실습으로 구현합니다. 실습은 '6일차\_모델성능최적화.ipynb' 파일에서 이어 진행합니다. 따라서 환경 준비나 데이터의 전처리 없이 모델 설계부터 시작하면 됩니다.

## 모델 설계

모델 선언을 다시 합니다. 모델 구조에는 변화가 없으므로 과적합 모델과 동일한 구조에 모델 이름만 model1로 변경합니다.

### CODE

```
n_feature = 28 * 28
n_class = 10
model1= nn.Sequential(nn.Flatten(),
                      nn.Linear(n_feature, 512),
                      nn.ReLU(),
                      nn.Linear(512, 256),
                      nn.ReLU(),
                      nn.Linear(256, 256),
                      nn.ReLU(),
                      nn.Linear(256, 64),
                      nn.ReLU(),
                      nn.Linear(64, n_class)
                      ).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model1.parameters(), lr=0.001)
```

모델을 설계했으니 이어서 조기 종료 코드를 구현합니다.

## 조기 종료 코드 구현

조기 종료를 위해서는 모델을 학습시키는 코드에서 몇 가지 조기 종료 장치를 추가해야 합니다. 모델을 학습시키는 코드를 다음과 같이 수정하면 됩니다.

### CODE

```
# 1. 조기 종료를 위한 변수 설정 -----
patience = 5           ①
best_loss = float('inf') ②
counter = 0             ③

tr_loss_list, val_loss_list = [], []
for t in range(40):
    tr_loss = train(train_loader, model1, loss_fn, optimizer, device)
    val_loss, pred = evaluate(x_val, y_val, model1, loss_fn, device)
    pred = nn.functional.softmax(pred, dim=1)
    pred = np.argmax(pred.cpu().numpy(), axis = 1)
    tr_loss_list.append(tr_loss)    # train - CrossEntropy
    val_loss_list.append(val_loss)  # val - CrossEntropy
    print(f'Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}')
```

```

# 2. 조기 종료 확인 -----
if val_loss > best_loss: ④
    counter += 1
    print(f'--> status, best_loss : {best_loss:4f}, counter : {counter}')
else:
    best_loss = val_loss ⑤
    counter = 0          ⑥

# 3. 조기 종료 조건 확인 -----
if counter >= patience: ⑦
    print("Early Stopping!")
    break ⑧

```

#### OUTPUT

```

Epoch 1, train loss : 0.7711, val loss : 0.5400
Epoch 2, train loss : 0.4830, val loss : 0.4596
Epoch 3, train loss : 0.4130, val loss : 0.4239
Epoch 4, train loss : 0.3768, val loss : 0.4157
Epoch 5, train loss : 0.3459, val loss : 0.4156
Epoch 6, train loss : 0.3217, val loss : 0.4078
Epoch 7, train loss : 0.3042, val loss : 0.4221
--> status, best_loss : 0.407803, counter : 1
Epoch 8, train loss : 0.2875, val loss : 0.4261
--> status, best_loss : 0.407803, counter : 2
Epoch 9, train loss : 0.2761, val loss : 0.4256
--> status, best_loss : 0.407803, counter : 3
Epoch 10, train loss : 0.2590, val loss : 0.4575
--> status, best_loss : 0.407803, counter : 4
Epoch 11, train loss : 0.2479, val loss : 0.4698
--> status, best_loss : 0.407803, counter : 5
Early Stopping!

```

- ① 오차가 이전보다 커지면 다섯 번까지 기다리도록 임계값을 지정합니다.
- ② 현재 에포크 시점에서 최적 오차(best\_loss)를 저장합니다(초깃값은 무한대로 설정).
- ③ 현재까지 몇 번 기다렸는지 카운트 값을 저장합니다.
- ④ 현재 오차가 최적 오차보다 크다면 카운터를 증가시킵니다.
- ⑤ 현재 오차가 최적 오차보다 작다면 현재 오차를 베스트 오차로 저장합니다.
- ⑥ 카운터를 리셋합니다.
- ⑦ 조기 종료 조건(5번 이상 기다렸는지)을 확인합니다.
- ⑧ 조기 종료 조건을 만족했으므로 학습을 멈춥니다.

best\_loss의 초깃값을 무한대로 설정한 까닭은 학습이 시작되었을 때 어떤 검증 오차가 나오더라도 처음에는 반드시 best\_loss보다 작게 만들기 위함입니다. 즉, 첫 번째 에포크에서 계산된 val\_loss가 어떤 값이든 무한대보다는 작기 때문에 자동으로 best\_loss로 저장되며, 이 best\_loss로 이후 에포크에서 검증 오차와 비교할

수 있습니다.

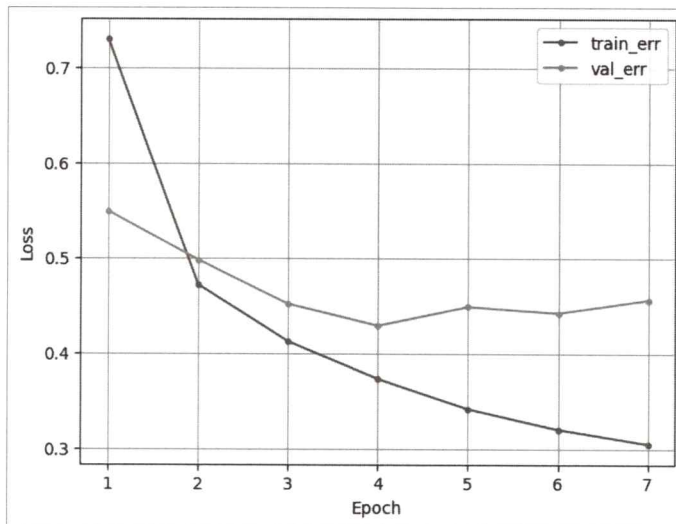
출력 결과를 보면 에포크 7에서 카운터가 1이 됩니다. 최적 오차(best\_loss)가 0.4078인데, 0.4221로 증가했기 때문입니다. 다음 에포크에서도 카운터가 1만큼 증가합니다. 검증 오차가 0.4261로 더 커졌습니다. 이후 지속적으로 검증 오차가 증가하다가 에포크 11에서 임계값 5에 도달했습니다. 종료 조건을 만족하므로 break 문을 만나 학습은 종료됩니다.

조기 종료된 모델의 학습 곡선을 그립니다.

CODE

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

OUTPUT



코드를 실행하면 조기 종료된 모델의 학습 곡선이 출력됩니다. 에포크 6에서 최적 오차를 기록하고 이후 에포크에서 검증 오차가 5번 증가해 조기 종료한 상태를 보여 줍니다.

#### 예측 및 평가 결과

조기 종료 모델의 예측 및 평가도 살펴보겠습니다. 빈 셀에서 다음 코드를 작성합니다.

**CODE**

```
# 예측하고 소프트맥스 함수 적용
_, pred = evaluate(x_test, y_test, model1, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)

# 가장 확률이 높은 인덱스 저장
pred = np.argmax(pred.cpu().numpy(), axis = 1)

# 모델 평가
print(classification_report(y_test, pred, target_names=class_list, digits=4))
```

**OUTPUT**

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| T-shirt/top  | 0.8679    | 0.7465 | 0.8026   | 493     |
| Trouser      | 0.9919    | 0.9461 | 0.9684   | 519     |
| Pullover     | 0.6709    | 0.8727 | 0.7586   | 479     |
| Dress        | 0.9420    | 0.8120 | 0.8722   | 500     |
| Coat         | 0.7528    | 0.6994 | 0.7251   | 479     |
| Sandal       | 0.9573    | 0.9573 | 0.9573   | 515     |
| Shirt        | 0.6528    | 0.6969 | 0.6741   | 518     |
| Sneaker      | 0.9258    | 0.9480 | 0.9368   | 500     |
| Bag          | 0.9492    | 0.9852 | 0.9669   | 474     |
| Ankle boot   | 0.9627    | 0.9388 | 0.9506   | 523     |
| accuracy     |           |        | 0.8608   | 5000    |
| macro avg    | 0.8673    | 0.8603 | 0.8613   | 5000    |
| weighted avg | 0.8686    | 0.8608 | 0.8622   | 5000    |

실행 결과를 보면 조금 의아할 수도 있습니다. 검증 오차(val\_loss)가 높은 상태인 과적합 모델과 검증 오차(val\_loss)가 최저점 근방에서 멈춘, 즉 검증 오차 성능 관점에서는 더 나은 모델의 예측 평가 결과가 거의 비슷하거나 오히려 더 떨어진 것처럼 보이기 때문입니다. 정분류율을 보면 과적합 모델은 0.8684인데, 조기 종료 모델은 0.8608입니다. 이 현상은 어떻게 해석해야 할까요?

**조기 종료를 적용할 때 고려할 사항**

조기 종료는 일반적으로 과적합을 방지하고 모델의 일반화 성능을 높이기 위해 사용하는 매우 유용한 기법입니다. 하지만 모든 상황에서 성능이 개선되는 것은 아닙니다. 실제로 검증 오차 기준으로 더 좋은 모델임에도 정분류율(Accuracy)에서는 과적합 모델과 큰 차이가 없거나 오히려 조금 낮게 나오는 경우도 있습니다.

이러한 현상은 보통 다음 두 가지 이유로 설명할 수 있습니다.

- 조기 종료 시점이 너무 이르면 학습이 충분히 진행되지 못한 상태일 수 있습니다.
- 검증 오차와 정분류율은 서로 다른 기준으로 평가하기 때문에 검증 오차의 최저점이 정분류율의 최고점과 꼭 일치하지는 않습니다.

두 가지 이유를 좀 더 자세히 설명하겠습니다. 먼저 조기 종료의 임계값인 `patience` 값(얼마나 기다릴지)을 잘못 설정하면 학습을 너무 일찍 끝내거나 과적합이 이미 진행된 후에 종료하는 문제가 생길 수 있습니다. 검증 오차의 변동 폭이 크다면 종료 시점 판단이 더 어렵습니다. 또한 조기 종료는 검증 오차(`val_loss`)가 연속적으로 나빠지면 학습을 중단합니다. 그런데 이 현상은 모델 학습이 일시적으로 정체된 상황이어서 조금 더 기다린다면 성능이 나아질 수도 있습니다. 즉, 너무 이른 종료는 학습 부족(Underfitting) 상태로 이어질 수 있습니다.

두 번째로 조기 종료는 보통 손실 함수(CrossEntropyLoss, BCELoss 등)를 기준으로 판단하지만, 모델의 성능을 평가할 때는 정분류율(Accuracy)처럼 예측이 실제 목표값과 일치했는지를 따지는 지표를 많이 사용합니다. 이 두 기준은 평가 방식이 다르기 때문에 손실값이 줄어든다고 해서 정확도도 반드시 함께 향상되는 것은 아닙니다.

예를 들어 이진 분류 문제에서 어떤 샘플의 예측 확률이 0.59에서 0.61로 바뀌었다고 가정해 봅시다. 손실 함수는 예측이 더 목표값 '1'에 가까워졌다고 보고 0.61일 때 더 작은 오차로 계산합니다. 하지만 두 확률 모두 0.5 이상이기 때문에 예측 범주는 '1'을 맞춘 결과로서 모델 성능은 평가됩니다. 두 예측 확률에 따른 정분류율에는 아무런 변화도 없습니다. 손실 함수는 예측의 '정도'까지 평가하는 반면, 정분류율은 단지 '맞았는지, 틀렸는지'만 따지기 때문에 이 같은 차이가 발생합니다.

한 가지 비유를 들어 설명하면 우리가 수학 문제 두 개를 풀었는데, 둘 다 틀렸다고 가정합시다. 첫 번째 문제는 개념 자체를 전혀 이해하지 못해 틀렸고, 두 번째 문제는 대부분 풀이 과정은 맞았지만 계산 실수로 틀렸습니다. 학습 과정이라면 두 문제의 틀림 정도에는 차이가 있습니다. 첫 번째는 완전히 틀린 것이고 두 번째는 거의 맞았다고 볼 수 있습니다. 학습에는 오류의 정도를 평가하는 것이 중요한데, 이 역할을 손실 함수가 합니다. 반면 시험은 두 문제 모두 '틀림'으로 처리하는데, 정분류율이나 정밀도 같은 성능 평가 지표는 단순히 맞았는지 틀렸는지 그 결과만을 판단하기 때문입니다.

이처럼 손실 함수와 성능 지표는 평가의 관점이 다르기 때문에 조기 종료의 판단 기준과 실제 모델의 성능 사이에는 차이가 발생할 수 있다는 점을 이해하고 적용해야 합니다.

이어서 과적합 방지를 위한 두 번째 방법인 드롭아웃(Dropout)을 알아봅니다.

## 모델 성능 최적화 실습 3: 드롭아웃

과적합을 방지하기 위한 또 다른 대표 기법의 하나가 드롭아웃(Dropout, 임의의 연결 끊기)입니다. 드롭아웃은 모델이 학습하는 동안 일부 노드를 임의로 비활성화해 모델이 특정 노드에 과도하게 의존하는 것을 방지하는 방법입니다.

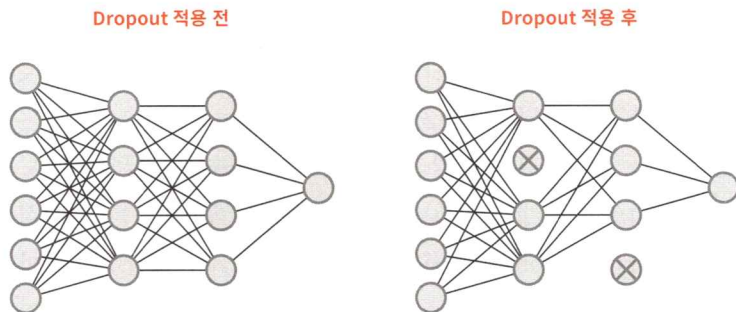
### 드롭아웃의 개념 이해

드롭아웃이 적용되는 과정은 다음과 같습니다.

1. 노드 비활성화: 학습할 때 무작위로 선택된 일부 노드를 제외합니다. 노드를 제외시키면 특정 노드에 대한 과도한 의존성을 방지할 수 있습니다(노드를 비활성화하거나 제외한다 함은 실제 연산에서 노드의 출력을 0으로 만든다는 의미입니다).
2. 순전파 및 역전파: 비활성화 노드는 학습 배치에서 계산에 참여하지 않으므로 순전파 및 역전파에서 제외됩니다.
3. 하나의 에포크에서 배치 단위마다 새롭게 무작위로 비활성화: 배치 단위마다 노드들이 무작위로 비활성화되므로 다양한 조합의 노드로 구성된 네트워크가 학습에 참여합니다.

드롭아웃 적용 전에는 모든 노드가 활성화된 상태에서 학습이 진행되므로 특정 노드나 경로에 대한 의존성이 높아질 수 있습니다. 그 결과 모델이 과적합할 가능성도 커집니다. 그러나 드롭아웃 적용 후에는 임의로 일부 노드가 비활성화되면서 특정 노드의 의존도가 낮아집니다. 이는 학습의 다양성을 높이고 더 안정적인 일반화 성능을 보장합니다.

[그림 6-7]은 드롭아웃 적용 전후의 노드 구성을 보여 줍니다.



[그림 6-7] 드롭아웃 적용 전과 후 비교

## 드롭아웃 구현

드롭아웃의 주요 하이퍼파라미터는 드롭아웃 비율(Dropout Rate)입니다. 드롭아웃 비율은 학습할 때 노드를 비활성화하는 비율입니다. 예를 들어 드롭아웃 비율이 0.4라면 학습 과정에서 각 은닉층별로 무작위로 선택된 40%의 노드가 학습의 배치마다 비활성화됩니다. 보통 드롭아웃 비율은 0.2~0.5 범위로 설정합니다.

## 드롭아웃 비율 설정

드롭아웃은 과적합을 방지하는 데 매우 효과적인 기법이지만, 얼마나 많은 노드를 비활성화할지, 즉 드롭아웃 비율을 어떻게 설정할지는 모델의 구조에 따라 달라집니다. 너무 낮은 비율은 효과가 미미할 수 있고, 너무 높으면 오히려 학습이 제대로 되지 않아 성능이 떨어질 수 있습니다. 따라서 다음 가이드를 참조해 비율을 설정하는 것이 좋습니다.

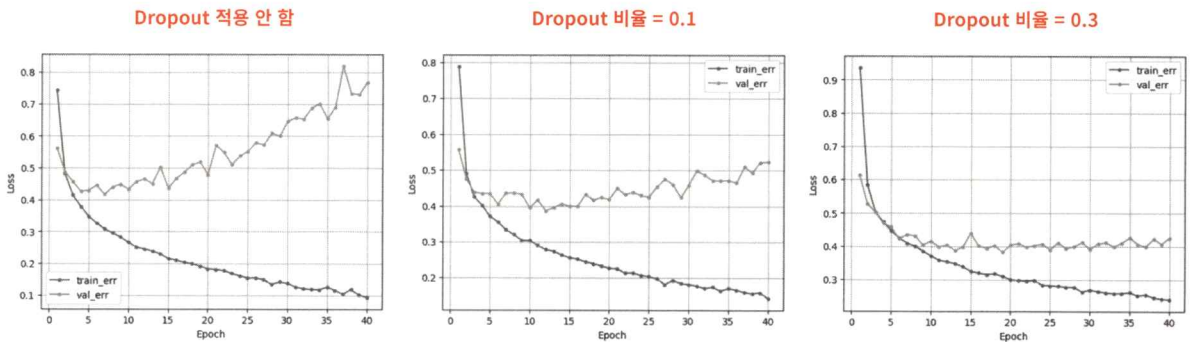
- 특징(feature) 수가 적을 경우: 이 경우에는 드롭아웃 비율을 너무 높이면 모델이 핵심적인 정보조차 놓치게 되어 학습 성능이 저하될 수 있습니다. 특징이 적다는 것은 모델이 학습에서 사용할 수 있는 정보가 제한적이라는 뜻입니다. 따라서 이럴 때는 비율을 낮게 설정하는 것이 좋습니다(권장 범위: 0.2~0.3).
- 특징(feature) 수가 많은 경우: 이 경우에는 모델이 불필요하거나 중복된 정보를 학습할 가능성이 높아집니다. 이때 드롭아웃을 적절히 적용하면 불필요한 특징에 대한 의존을 줄이고 더 일반화된 패턴을 학습할 수 있습니다. 따라서 비율을 조금 높게 설정합니다(권장 범위: 0.4~0.5)
- 은닉층이 적은 경우(얇은 네트워크): 은닉층이 적은 모델은 파라미터 수가 적고

복잡도도 비교적 낮습니다. 이 경우에는 과적합의 위험이 크지 않기 때문에 드롭아웃을 과도하게 적용할 필요가 없습니다. 따라서 드롭아웃 비율을 낮추는 것이 적절합니다(권장 범위: 0.2~0.3).

- 은닉층이 많은 경우(깊은 네트워크): 은닉층이 많을수록 모델의 복잡도가 증가하고 과적합의 위험도 함께 커집니다. 특히 수많은 파라미터를 가진 심층 신경망(Deep Neural Network)은 학습 데이터에 지나치게 적합할 가능성이 높습니다. 이때는 드롭아웃을 보다 적극적으로 적용해 특정 경로나 노드의 의존을 줄이는 것이 필요합니다(권장 범위: 0.4~0.5)

제시한 권장 범위는 참조 사항일 뿐입니다.

[그림 6-8]은 드롭아웃 비율에 따른 학습 곡선의 차이를 보여 줍니다.



[그림 6-8] 드롭아웃 비율에 따른 학습 곡선 비교

[그림 6-8]의 가운데 그래프에서 드롭아웃 비율을 0.1로 지정하면 과적합 현상이 확연히 줄어듭니다. 비율을 0.3으로 더 올리면 에포크 수가 증가해도 더 이상 과적합이 발생하지 않습니다.

드롭아웃 비율은 모델 복잡도, 데이터의 양, 특징 수에 따라 달리 조정되어야 하므로 정해진 정답은 없습니다. 역시 실험으로 최적값을 찾아야 합니다.

## 드롭아웃 코드 구현

앞에서 사용했던 과적합 모델 구조에 드롭아웃(Dropout) 기법을 적용합니다. 빈 셀에서 모델의 은닉층에 드롭아웃을 삽입한 model2를 선언합니다.

**CODE**

```

n_feature = 28 * 28
n_class = 10
dropout_rate = 0.4 ①
model2 = nn.Sequential(nn.Flatten(),
                        nn.Linear(n_feature, 512), nn.ReLU(),
                        nn.Dropout(dropout_rate),
                        nn.Linear(512, 256), nn.ReLU(),
                        nn.Dropout(dropout_rate),
                        nn.Linear(256, 256), nn.ReLU(), ②
                        nn.Dropout(dropout_rate),
                        nn.Linear(256, 64), nn.ReLU(),
                        nn.Dropout(dropout_rate),
                        nn.Linear(64, n_class)
                    ).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model2.parameters(), lr=0.001)

```

① 드롭아웃 비율을 0.4로 지정합니다.

② 각 은닉층의 nn.Linear와 nn.ReLU 함수를 적용한 다음, nn.Dropout 함수를 적용해 해당 층에서 무작위로 노드를 끊도록 구성합니다.

보통 드롭아웃은 은닉층과 활성화 함수 다음에 위치하는 것이 보통이며 출력층에는 적용하지 않습니다. 대부분 은닉층마다 동일한 드롭아웃 비율을 설정하지만, 네트워크의 구조나 실험 목적에 따라 층별로 서로 다른 비율을 지정할 수도 있습니다.

이어서 학습 및 예측 평가를 수행합니다.

**학습**

모델의 학습 코드는 동일합니다. 40번의 에포크 동안 학습을 진행합니다.

**CODE**

```

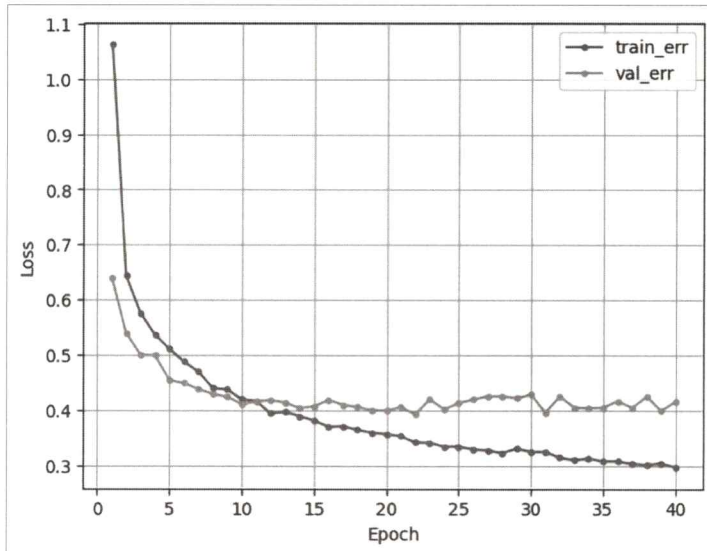
epochs = 40
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model2, loss_fn, optimizer, device)
    val_loss, pred = evaluate(x_val, y_val, model2, loss_fn, device)
    # 리스트에 추가
    tr_loss_list.append(tr_loss)    # train - CrossEntropy
    val_loss_list.append(val_loss)  # val - CrossEntropy
    print(f"Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}")

dl_learning_curve(tr_loss_list, val_loss_list)

```

#### OUTPUT

```
Epoch 1, train loss : 1.0636, val loss : 0.6392
Epoch 2, train loss : 0.6428, val loss : 0.5399
Epoch 3, train loss : 0.5759, val loss : 0.5001
...
Epoch 38, train loss : 0.3012, val loss : 0.4250
Epoch 39, train loss : 0.3039, val loss : 0.3992
Epoch 40, train loss : 0.2971, val loss : 0.4160
```



학습 곡선을 보면 에포크가 증가하더라도 검증 오차 그래프는 완만하게 유지되고 있습니다. 이 부분이 드롭아웃의 효과입니다.

#### 예측 및 평가

이제 드롭아웃 모델의 성능을 확인합니다. 다음과 같이 코드를 작성합니다.

#### CODE

```
# 모델 예측
_, pred = evaluate(x_test, y_test, model2, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)
pred = np.argmax(pred.cpu().numpy(), axis = 1)

# 모델 평가
print(classification_report(y_test.numpy(), pred, target_names=class_list,
digits=4))
```

# OUTPUT

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| T-shirt/top  | 0.7882    | 0.8682 | 0.8263   | 493     |
| Trouser      | 0.9861    | 0.9557 | 0.9706   | 519     |
| Pullover     | 0.8062    | 0.7557 | 0.7802   | 479     |
| Dress        | 0.8836    | 0.8960 | 0.8898   | 500     |
| Coat         | 0.6974    | 0.8518 | 0.7669   | 479     |
| Sandal       | 0.9775    | 0.9282 | 0.9522   | 515     |
| Shirt        | 0.7343    | 0.5656 | 0.6390   | 518     |
| Sneaker      | 0.8815    | 0.9820 | 0.9290   | 500     |
| Bag          | 0.9685    | 0.9726 | 0.9705   | 474     |
| Ankle boot   | 0.9715    | 0.9140 | 0.9419   | 523     |
| accuracy     |           |        | 0.8686   | 5000    |
| macro avg    | 0.8695    | 0.8690 | 0.8666   | 5000    |
| weighted avg | 0.8708    | 0.8686 | 0.8671   | 5000    |

정분류를 관점에서 보면 과적합 모델(0.8684)과 드롭아웃을 적용한 모델(0.8686)은 성능에서 차이가 거의 없습니다. 그러나 학습 과정에서 검증 오차(CrossEntropy Loss)를 비교하면 두 모델 사이에 분명한 차이가 나타납니다. 앞에서 만든 과적합 모델의 학습 결과에서 최종 에포크의 검증 오차(val\_loss)는 0.7337이었습니다. 반면 드롭아웃을 적용한 모델에서 최종 에포크의 검증 오차는 0.4160으로 과적합 모델의 결과보다 훨씬 낮습니다.

마치 시험 점수는 비슷하지만, 한 문제를 풀 때마다 드롭아웃 모델이 더 정확히 풀었다는 의미입니다. 즉, 드롭아웃 모델은 문제를 더 안정적으로 풀 수 있는 능력을 갖추고 있으며, 새롭거나 응용된 문제 유형이 나왔을 때도 더 잘 대응할 가능성이 높음을 시사합니다. 반면 과적합 모델은 학습된 문제와 그와 유사한 문제에는 익숙하지만, 새로운 문제에서는 취약합니다.

## 모델 성능 최적화 실습 4: 가중치 규제

모델의 복잡도를 조절해 과적합을 방지하는 또 다른 중요한 방법은 가중치 규제(Weight Regularization)입니다. 가중치 규제는 손실 함수에 페널티 항(Regularization Term)을 더해 특정 가중치가 지나치게 커지지 않도록 제어하는 기법입니다. 가중치 규제는 모델이 일부 입력 특징에 과도하게 반응하는 것을 막고 보다 안정적으로 학습할 수 있도록 돕습니다.

또한 가중치 규제를 적용하면 불필요하게 작은 가중치들은 자연스럽게 0에 가까

워져 모델이 중요 특징에 집중하도록 구조를 단순화하는 효과도 얻을 수 있습니다. 구조가 단순해지면 모델은 더 간결해지고 새 데이터에서의 일반화 능력이 더욱 향상됩니다.

### 가중치가 커지면 무엇이 문제가 되는가?

딥러닝이나 머신러닝 모델에서 가중치(Weight)는 입력이 출력에 얼마나 영향을 미치는지 결정하는 핵심 요소입니다. 하지만 가중치 값이 지나치게 커지면 다음과 같은 문제가 발생할 수 있습니다.

- 모델이 과적합(Overfitting)할 가능성 증가: 가중치가 크면 학습 데이터에 지나치게 민감해져 새로운 데이터에서 일반화 성능이 떨어집니다.
- 학습 불안정성: 가중치 값이 커지면 기울기(Gradient)도 커져 학습이 불안정해질 수 있습니다. 이는 기울기 폭주(Gradient Explosion) 문제로 이어질 수 있습니다.
- 모델 해석의 어려움: 가중치 값이 너무 크면 특정 입력에 의존도를 과도하게 높이는 결과를 초래하므로 모델 해석을 어렵게 만듭니다.

대표적인 가중치 규제 방법으로 L1 규제(L1 Regularization)와 L2 규제(L2 Regularization)가 있습니다. 두 방법 모두 손실 함수에서 가중치 크기에 따라 페널티 항을 추가해 모델의 가중치가 불필요하게 커지지 않도록 제어합니다. 다만 적용 방식과 효과에서 차이가 있습니다. L1 규제는 가중치 절댓값의 합을 페널티로 사용해 일부 가중치를 0으로 만들어 특징 선택(Feature Selection) 효과를 내는 데 유리합니다. 반면 L2 규제는 가중치의 제곱합을 페널티로 사용하며, 큰 가중치를 더 강하게 억제해 가중치를 부드럽게 분포하도록 만들기 때문에 과적합을 완화하는 데 효과적입니다.

실무에서는 딥러닝 모델의 안정적인 학습을 위해 L2 규제가 기본값처럼 널리 사용됩니다. 따라서 이번 절에서는 L1, L2 규제의 개념과 코드를 모두 설명하지만, 여러분은 L2 규제를 위주로 사용하면 됩니다.



#### L1, L2의 의미

L은 Norm(노름)을 나타내는 수학적인 기호입니다. 노름은 벡터의 크기(길이)를 특정 방식으로 측정하는 수학 연산입니다.

- L1 노름: 1차 노름이라고 하며, 벡터 각 요소의 절댓값을 모두 더한 값으로 길이를 측정합니다.
- L2 노름: 2차 노름이라고 하며, 벡터 각각의 요소를 제곱한 다음 합하고 제곱근을 취해 길이를 측정합니다.

여기서 숫자 1, 2는 “몇 차수의 노름을 사용했는가”를 의미합니다. L은 ‘Length’에서 온 기호입니다.

## L2 규제

### L2 규제란?

L2 규제(Regularization)는 릿지(Ridge)라고도 하며 오차에 가중치의 제곱합을 더해 계산합니다. 가중치 값이 너무 커지지 않도록 제어합니다. L2 규제가 포함된 손실 함수는 다음과 같습니다.

$$\text{Loss}_{\text{total}} = \text{Loss}_{\text{original}} + \lambda \cdot \sum w^2$$

- $\lambda$ : 규제 강도
- $w$ : 각각의 가중치 값

기존의 손실 함수에  $\lambda \cdot \sum w^2$ 가 추가되어 있는데, 이 항이 바로 L2 규제입니다. L2 규제는 가중치가 0에 가깝게 축소되도록 유도하지만, 대다수 가중치는 0이 되지 않고 작게 유지됩니다. 모델이 과도하게 복잡해지는 것을 막아 과적합을 줄이는 용도로 사용됩니다.

람다( $\lambda$ )는 규제를 얼마나 세게 가할지 강도를 조절하는 하이퍼파라미터입니다.

- $\lambda$ 가 작으면 규제 효과가 약해져 가중치가 거의 줄지 않습니다(과적합).
- $\lambda$ 가 너무 크면 많은 가중치가 0이 되어 모델의 성능이 떨어질 수 있습니다(과소적합).

L2 규제에서 람다의 일반적인 범위는  $1e-5 \sim 1e-2$  사이입니다. 하지만 가장 적절한  $\lambda$  값을 찾기 위해서는 하이퍼파라미터 튜닝이 반드시 필요합니다.

## L2 규제 코드 적용하기

파이토치에서는 옵티마이저의 `weight_decay` 옵션으로 L2 규제를 쉽게 적용할 수 있습니다.

앞서 사용했던 과적합 모델 구조에 L2 규제 기법을 적용합니다. 모델의 옵티마이저를 수정한 `model3`을 선언합니다.

### CODE

```
n_feature = 28 * 28
n_class = 10
model3 = nn.Sequential(nn.Flatten(),
                        nn.Linear(n_feature, 512),
                        nn.ReLU(),
                        nn.Linear(512, 256),
                        nn.ReLU(),
                        nn.Linear(256, 256),
                        nn.ReLU(),
                        nn.Linear(256, 64),
                        nn.ReLU(),
                        nn.Linear(64, n_class)
                        ).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model3.parameters(), lr=0.001, weight_decay = 0.001) ①
```

① `weight_decay` 옵션에서 지정한 값이 바로 L2 규제의 강도입니다.

모델을 변경했다면 에포크 수를 40으로 지정해 학습시킵니다.

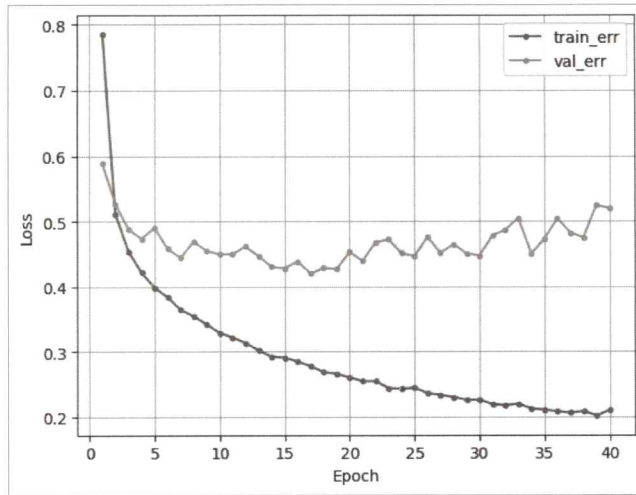
### CODE

```
epochs = 40
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model3, loss_fn, optimizer, device)
    val_loss, pred = evaluate(x_val, y_val, model3, loss_fn, device)
    tr_loss_list.append(tr_loss) # train - CrossEntropy
    val_loss_list.append(val_loss) # val - CrossEntropy
    print(f"Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}")

dl_learning_curve(tr_loss_list, val_loss_list)
```

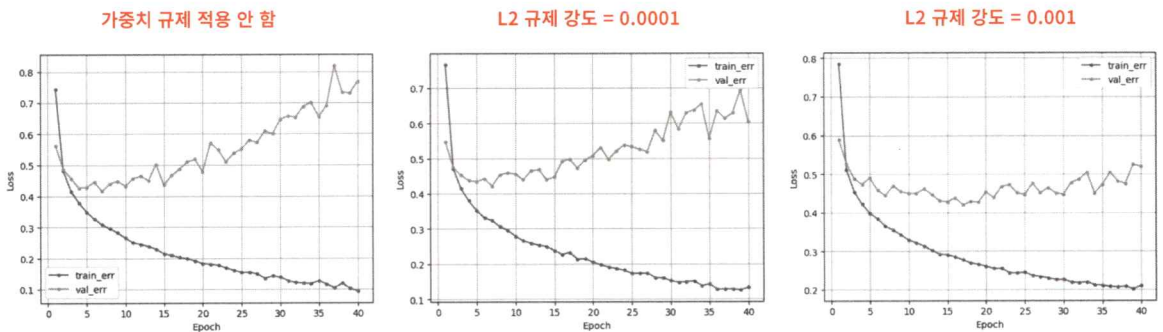
### OUTPUT

```
Epoch 1, train loss : 0.7850, val loss : 0.5881
Epoch 2, train loss : 0.5112, val loss : 0.5269
Epoch 3, train loss : 0.4539, val loss : 0.4877
...
Epoch 38, train loss : 0.2085, val loss : 0.4748
Epoch 39, train loss : 0.2018, val loss : 0.5240
Epoch 40, train loss : 0.2101, val loss : 0.5192
```



실행 결과를 살펴보면 검증 오차가 줄다가 조금 올라가긴 하지만, 과적합 모델처럼 급격히 상승하지는 않습니다. 가중치를 규제하기 때문입니다.

[그림 6-9]는 규제 강도에 따라 학습 곡선이 어떻게 달라지는지 비교한 그래프입니다. 규제 강도가 0.0001일 때는 과적합의 완화가 미세하지만, 0.001일 때는 큰 편입니다.



[그림 6-9] L2 규제 강도에 따른 학습 곡선 비교

학습을 완료한 가중치 규제 모델로 예측하고 평가합니다.

#### CODE

```
# 모델 예측
_, pred = evaluate(x_test, y_test, model3, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)
pred = np.argmax(pred.numpy(), axis = 1)
```

```
# 모델 평가
print(classification_report(y_test.numpy(), pred, target_names=class_list, digits=4))
```

#### OUTPUT

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| T-shirt/top  | 0.8747    | 0.7221 | 0.7911   | 493     |
| Trouser      | 1.0000    | 0.9383 | 0.9682   | 519     |
| Pullover     | 0.7401    | 0.8205 | 0.7782   | 479     |
| Dress        | 0.9142    | 0.8100 | 0.8590   | 500     |
| Coat         | 0.6995    | 0.8309 | 0.7595   | 479     |
| Sandal       | 0.9328    | 0.9437 | 0.9382   | 515     |
| Shirt        | 0.6346    | 0.6873 | 0.6599   | 518     |
| Sneaker      | 0.8603    | 0.9480 | 0.9020   | 500     |
| Bag          | 0.9719    | 0.9494 | 0.9605   | 474     |
| Ankle boot   | 0.9636    | 0.8604 | 0.9091   | 523     |
| accuracy     |           |        | 0.8510   | 5000    |
| macro avg    | 0.8592    | 0.8511 | 0.8526   | 5000    |
| weighted avg | 0.8602    | 0.8510 | 0.8531   | 5000    |

정분류율이 0.8510인데, 다른 모델에 비해 조금 떨어지는 것으로 나옵니다. 하지만 성능의 차이가 크다고 볼 수는 없습니다.

이어서 L1 규제도 살펴보겠습니다.

## L1 규제

### L1 규제란?

L1 규제는 라쏘(Lasso)라고도 하며 오차에 가중치 절댓값의 합을 더해 가중치 값이 너무 커지지 않도록 제어합니다. L1 규제가 포함된 손실 함수는 다음과 같습니다.

$$\text{Loss}_{\text{total}} = \text{Loss}_{\text{original}} + \lambda \cdot \sum |w|$$

- $\lambda$ : 규제 강도
- $w$ : 각각의 가중치 값

L1 규제는 손실 함수에 가중치 절댓값의 합을 페널티로 추가하는 방식입니다. 이 방식은 일부 가중치를 아예 0으로 만드는 성질이 있습니다. 즉, 모델이 학습 과정에서 덜 중요한 입력 특징(feature)이라고 판단하면 가중치를 0으로 만들어 해당 특징을 무시합니다. 따라서 불필요한 특징을 자동으로 제거하는 효과, 즉 특징 선택 (Feature Selection)을 합니다.

L1 규제에서 람다의 일반적인 범위는 0~0.1 사이입니다. L2 규제와 마찬가지로 규제 강도에 따라 과적합의 완화 정도가 달라집니다.

### 코드 적용하기

L1 규제는 L2 규제와 달리 학습(train) 코드를 직접 수정해야 합니다. 일반적으로 L2 규제를 많이 사용하기 때문에 옵션으로 L1 규제를 쉽게 적용하는 코드를 지원하지 않습니다. 여기서는 예제 코드로 간략하게만 설명하니 참조하길 바랍니다.

L1 규제는 학습 코드에서 일부 수정이 필요한 반면, 모델 구조나 손실 함수, 옵티마이저 설정은 이전과 동일합니다. 사용자 정의 함수인 train 함수를 다음과 같이 수정합니다.

```
CODE
def train(dataloader, model, loss_fn, optimizer, device, lambda_l1=1e-5):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    tr_loss = 0

    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # Forward pass
        pred = model(X)
        loss = loss_fn(pred, y)

        # L1 규제 항 계산 및 추가 -----
        l1_penalty = 0 ①
        for param in model.parameters(): ②
            l1_penalty += torch.sum(torch.abs(param)) ③
        loss += lambda_l1 * l1_penalty ④
        # -----
        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        tr_loss += loss.item()
    tr_loss /= num_batches
    return tr_loss
```

① 초기화: 전체 모델의 L1 페널티(규제 항)를 저장할 변수를 0으로 초기화합니다. 이 변수에 각 파라미터의 절댓값 합을 누적할 예정입니다.

② 파라미터 순회: 모델에서 학습 가능한 모든 파라미터(가중치와 편향 등)를 하나씩 순회합니다.

다. `parameters` 메서드는 파이토치 모델의 모든 파라미터 객체(`torch.nn.Parameter`)를 반환합니다.

- ③ 절댓값 합산: `torch.abs(param)`은 텐서로 된 해당 파라미터의 모든 요소에서 절댓값을 계산합니다. `torch.sum` 메서드는 절댓값 텐서의 모든 요소를 합산해 하나의 스칼라 값을 반환합니다. 이 값을 `l1_penalty`에 더합니다. 즉, 모든 파라미터의 절댓값 총합을 계산합니다.
- ④ L1 페널티를 오차에 추가: 지금까지 계산한 `l1_penalty`에 규제 강도 계수 `lambda_l1`을 곱합니다. 이렇게 만든 규제 항을 기존 오차(`loss`)에 더합니다.

즉, 모든 파라미터의 절댓값을 계산해 그 합을 오차에 더하는 부분이 추가되었습니다. 이것이 바로 L1 규제(라쏘 규제)이며 과적합을 방지하는 데 도움을 줍니다. L1 규제를 적용한 모델을 선언하고 학습시키고 평가하는 작업은 여러분이 직접 해보기를 바랍니다.

## 정리

이번 차시에서는 딥러닝 모델의 복잡도와 일반화 성능 사이의 균형을 어떻게 조절할 수 있는지를 학습했습니다. 모델이 지나치게 단순하면 학습 데이터의 패턴을 제대로 학습하지 못해 과소적합(Underfitting)이 발생하고, 반대로 지나치게 복잡하면 학습 데이터에 과도하게 맞춰져 새로운 데이터에서 성능이 떨어지는 과적합(Overfitting) 문제가 발생할 수 있습니다. 따라서 모델의 복잡도를 적절히 조절하는 것이 매우 중요합니다.

복잡도를 조절하기 위한 대표적인 방법으로는 조기 종료(Early Stopping), 드롭아웃(Dropout), 가중치 규제(Regularization)가 있습니다. 조기 종료는 검증 오차가 더 이상 개선되지 않을 때 학습을 중단해 과적합을 방지하는 기법입니다. 드롭아웃은 학습 과정에서 일부 노드를 무작위로 비활성화합니다. 이를 통해 모델이 특정 노드에 과도하게 의존하는 현상을 줄여 주어 모델의 일반화 성능을 높입니다. 가중치 규제는 손실 함수에 페널티 항을 추가해 가중치가 지나치게 커지지 않도록 억제하고 모델을 간결하게 만들어 안정적인 학습을 유도합니다. 특히 L1 규제는 불필요한 가중치를 0으로 만들어 특징 선택 효과를 제공하는 반면, L2 규제는 전체 가중치를 적절히 줄여 모델의 일반화 성능을 높이는 데 뛰어납니다.

[표 6-1]은 이번 차시에서 실험한 4가지 모델의 성능을 상호 비교한 표입니다.

[표 6-1] 실험별 성능 평가 결과 비교

| 성능                      | 과적합 모델 | 조기 종료  | 드롭아웃   | 가중치 규제 |
|-------------------------|--------|--------|--------|--------|
| 검증 오차(CrossEntropyLoss) | 0.7337 | 0.4698 | 0.416  | 0.5192 |
| 정분류율(Accuracy)          | 0.8686 | 0.8608 | 0.8686 | 0.8510 |
| 최종 에포크 수                | 40     | 11     | 40     | 40     |

[표 6-1]로 확인할 수 있듯이 드롭아웃 모델이 검증 오차가 가장 낮고 정분류율도 높은 수준을 유지하면서 가장 안정적인 성능을 나타내고 있습니다. 조기 종료는 학습 횟수(에포크 수)를 줄이면서도 과적합 모델보다 낮은 검증 오차를 유지합니다. 가중치 규제 또한 검증 오차를 줄이는 데 효과적이지만, 정분류율 면에서는 다른 모델에 비해 상대적으로 낮은 성능을 보입니다.

이와 같이 복잡도 조절 기법들은 모델이 새로운 데이터에서 안정적인 예측 성능을 유지하도록 돕는 전략입니다. 실무에서는 조절 기법 하나만을 단독으로 사용하는 것이 아니라, 반복적인 실험 과정에서 여러 기법을 조합(예: 조기 멈춤+드롭아웃)해 사용하면서 최적의 성능을 탐색하는 일이 일반적입니다.

다음 차시에서는 모델 구조를 정의할 때 자주 사용하는 클래스 선언 방식을 알아보겠습니다.